Jiashuo Wang(364341)- DBSAssignment4

A- In my database design, I chose to create the following domains to ensure data consistency and avoid code duplication:

1. title_type: Both the exhibition and artwork tables have a title attribute with the exact same constraint (a minimum of 2 characters and a maximum of 40 characters). By creating a domain VARCHAR(40) CHECK (LENGTH(VALUE) >= 2 AND LENGTH(VALUE) <= 40), I applied this rule globally without repeating the CHECK constraint in multiple tables.
2. artwork_id_type: The artwork_id must be exactly 6 characters. I created a domain using CHAR(6) to strictly enforce this length, which is used in the artwork table as the Primary Key, and in junction tables like exhibition_artwork and artwork_artist as Foreign Keys. This guarantees that the data types of PKs and FKs perfectly match across the database.

B- Independent Tables (Created First):

I started by creating the tables that do not have any foreign keys pointing to other tables. These are visitor, gallery, artist, and artwork. The order among these four does not matter, as they are completely independent.

Tables with Single Dependencies (Created Second):

Next, I created the exhibition table. Concrete example: The exhibition table contains gallery_name as a Foreign Key. If I tried to create exhibition before gallery, the SQL compiler would throw an error because the referenced primary key does not yet exist. Therefore, exhibition must be created after gallery.

Junction/Associative Tables (Created Last):

Finally, I created the tables that resolve many-to-many relationships, as they depend on multiple tables. These include ticket, exhibition_artwork, and artwork_artist. Concrete example: The ticket table contains two foreign keys (visitor_id and exhibition_id). Thus, it could only be created at the very end, after both the visitor and exhibition tables were successfully created.

C-

- Ticket referencing exhibition (ON DELETE CASCADE):
  If an exhibition is completely canceled and deleted from the database, all the tickets purchased specifically for that exhibition become invalid and obsolete. Using ON DELETE CASCADE ensures that all related ticket records are automatically removed, preventing orphaned data.
- Exhibition referencing gallery (ON UPDATE CASCADE):
  The gallery_name is used as a primary key. Since names are descriptive text, a gallery might decide to rebrand or correct a typo in its name. By adding ON UPDATE CASCADE, the database will automatically update the gallery_name for all exhibitions hosted by that gallery, ensuring a smooth transition without breaking foreign key constraints.
- Exhibition_artwork referencing artwork (ON DELETE CASCADE):
  The junction table exhibition_artwork records which artworks are in which exhibitions. If a specific artwork is permanently removed from the association's system (deleted from the artwork table), it should logically be removed from all exhibition line-ups as well. ON DELETE CASCADE handles this automatically.

D- Handling Auto-incremented Primary Keys (SERIAL):

When inserting data into the visitor and exhibition tables, I intentionally omitted the visitor_id and exhibition_id columns from the INSERT statements. I relied on the database's SERIAL sequence to automatically generate these IDs starting from 1. Consequently, when inserting into the dependent tables like ticket and exhibition_artwork, I assumed these IDs were 1 and 2 for my foreign key references.

E-
```
CREATE SCHEMA IF NOT EXISTS s2_a4_art_galleries;
SET SCHEMA 's2_a4_art_galleries';

CREATE DOMAIN title_type AS VARCHAR(40) CHECK (LENGTH(VALUE) >= 2 AND
LENGTH(VALUE) <= 40);
CREATE DOMAIN artwork_id_type AS CHAR(6);

CREATE TABLE visitor (
    visitor_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50),
    visitor_type VARCHAR(10) NOT NULL CHECK (visitor_type IN ('Standard', 'VIP'))
);
CREATE TABLE gallery (
    gallery_name VARCHAR(100) PRIMARY KEY,
    address VARCHAR(200) NOT NULL CHECK (LENGTH(address) >= 12),
    opening_time TIME NOT NULL,
    closing_time TIME NOT NULL,
    CONSTRAINT valid_gallery_time CHECK (closing_time > opening_time)
);
CREATE TABLE artist (
    artist_name VARCHAR(100) PRIMARY KEY,
    years_of_experience SMALLINT NOT NULL CHECK (years_of_experience
BETWEEN 0 AND 99)
);
CREATE TABLE artwork (
    artwork_id artwork_id_type PRIMARY KEY,
    title title_type NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('Painting', 'Sculpture', 'Photo')),
    description VARCHAR(256)
);

CREATE TABLE exhibition (
    exhibition_id SERIAL PRIMARY KEY,
    title title_type NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    price DECIMAL(4,2) NOT NULL CHECK (price BETWEEN 10.00 AND 99.99),
```

```
    gallery_name VARCHAR(100) NOT NULL,
    CONSTRAINT valid_exhibition_date CHECK (end_date > start_date),
    FOREIGN KEY (gallery_name) REFERENCES gallery(gallery_name)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE ticket (
    visitor_id INTEGER NOT NULL,
    exhibition_id INTEGER NOT NULL,
    purchase_time TIMESTAMP NOT NULL,
    PRIMARY KEY (visitor_id, exhibition_id, purchase_time),
    FOREIGN KEY (visitor_id) REFERENCES visitor(visitor_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (exhibition_id) REFERENCES exhibition(exhibition_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE exhibition_artwork (
    exhibition_id INTEGER NOT NULL,
    artwork_id artwork_id_type NOT NULL,
    PRIMARY KEY (exhibition_id, artwork_id),
    FOREIGN KEY (exhibition_id) REFERENCES exhibition(exhibition_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (artwork_id) REFERENCES artwork(artwork_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE artwork_artist (
    artist_name VARCHAR(100) NOT NULL,
    artwork_id artwork_id_type NOT NULL,
    PRIMARY KEY (artist_name, artwork_id),
    FOREIGN KEY (artist_name) REFERENCES artist(artist_name)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (artwork_id) REFERENCES artwork(artwork_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);

--insert
-- 1. Insert into visitor
INSERT INTO visitor (first_name, last_name, visitor_type) VALUES
    ('John', 'Doe', 'Standard'),
```

```
    ('Jane', NULL, 'VIP');

-- 2. Insert into gallery
INSERT INTO gallery (gallery_name, address, opening_time, closing_time) VALUES
    ('Louvre Museum', 'Rue de Rivoli, Paris', '09:00:00', '18:00:00'),
    ('Aros Modern', 'Aros Alle 2, Aarhus', '10:00:00', '21:00:00');

-- 3. Insert into artist
INSERT INTO artist (artist_name, years_of_experience) VALUES
    ('Vincent van Gogh', 10),
    ('Leonardo da Vinci', 45);

-- 4. Insert into artwork
INSERT INTO artwork (artwork_id, title, type, description) VALUES
    ('VG0001', 'Starry Night', 'Painting', 'A beautiful night sky painting.'),
    ('LV0002', 'David', 'Sculpture', NULL);

-- 5. Insert into exhibition
INSERT INTO exhibition (title, start_date, end_date, price, gallery_name) VALUES
    ('Impressionism Era', '2026-05-01', '2026-08-31', 45.50, 'Louvre Museum'),
    ('Modern Classics', '2026-09-01', '2026-10-15', 25.00, 'Aros Modern');

-- 6. Insert into ticket
INSERT INTO ticket (visitor_id, exhibition_id, purchase_time) VALUES
    (1, 1, '2026-04-15 14:30:00'),
    (2, 2, '2026-08-20 09:15:00');

-- 7. Insert into exhibition_artwork
INSERT INTO exhibition_artwork (exhibition_id, artwork_id) VALUES
    (1, 'VG0001'),
    (2, 'LV0002');

-- 8. Insert into artwork_artist
INSERT INTO artwork_artist (artist_name, artwork_id) VALUES
    ('Vincent van Gogh', 'VG0001'),
    ('Leonardo da Vinci', 'LV0002');
```